COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	10/6/26	**DATE SUBMITTED:**	10/6/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A:

Graphs.cpp × Graphs.h ×

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>

struct adjNode {
    int val;
    int cost;
    adjNode* next;
};

struct graphEdge {
    int start_ver;
    int end_ver;
    int weight;
};

class DiaGraph {
private:
    int N;
    adjNode* getAdjListNode(int value, int weight, adjNode* head);

public:
    adjNode** head;
    DiaGraph(graphEdge edges[], int n, int N);
    ~DiaGraph();
    void displayAdjList(int i);
};
```

```cpp
#include <iostream>
#include "Graphs.h"

adjNode* DiaGraph::getAdjListNode(int value, int weight, adjNode* head) {
    adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;
    newNode->next = head;
    return newNode;
}

DiaGraph::DiaGraph(graphEdge edges[], int n, int N) {
    this->N = N;
    head = new adjNode*[N]();

    for (int i = 0; i < N; ++i) {
        head[i] = nullptr;
    }

    for (int i = 0; i < n; i++) {
        int start_ver = edges[i].start_ver;
        int end_ver = edges[i].end_ver;
        int weight = edges[i].weight;

        adjNode* newNode = getAdjListNode( value: end_ver, weight, head[start_ver]);
        head[start_ver] = newNode;
    }
}
```

```cpp
DiaGraph::~DiaGraph() {
    for (int i = 0; i < N; i++) {
        adjNode* current = head[i];
        while (current != nullptr) {
            adjNode* next = current->next;
            delete current;
            current = next;
        }
    }
    delete[] head;
}

void DiaGraph::displayAdjList(int i) {
    adjNode* ptr = head[i];
    while (ptr != nullptr) {
        std::cout << "(" << i << ", " << ptr->val << ", " << ptr->cost << ") ";
        ptr = ptr->next;
    }
    std::cout << std::endl;
}
```

```cpp
int main() {
    graphEdge edges[] = {
        { .start_ver: 0, .end_ver: 1, .weight: 2}, { .start_ver: 0, .end_ver: 2, .weight: 4},
        { .start_ver: 1, .end_ver: 4, .weight: 3}, { .start_ver: 2, .end_ver: 3, .weight: 2},
        { .start_ver: 3, .end_ver: 1, .weight: 4}, { .start_ver: 4, .end_ver: 3, .weight: 3}
    };

    int N = 6;
    int n = sizeof(edges) / sizeof(edges[0]);

    DiaGraph diagraph(edges, n, N);

    std::cout << "Graph adjacency list\n(start_vertex, end_vertex, weight):\n";
    for (int i = 0; i < N; i++) {
        diagraph.displayAdjList(i);
    }

    return 0;
}
```

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)
```

Code Analysis:
The program starts by defining a simple node structure that holds a target vertex, a weight cost, and a pointer to the next node. When a new connection is created, the code attaches it directly to the front of that vertex's list. Pushing nodes to the front takes O(1) constant time.

ILO B.1:

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Step 2: Edge struct
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```
};

// Step 3: Stream operator overload
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e :Edge<T> & : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

// Step 4: Graph class definition
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

    auto vertices() const ->size_t
    {
```

```
53      {
54          return V;
55      }
56
57      auto &edges() const ->const vector<Edge<T>> &
58      {
59          return edge_list;
60      }
61
62      void add_edge(Edge<T> &&e)
63      {
64          if (e.src >= 1 && e.src <= V &&
65              e.dest >= 1 && e.dest <= V)
66              edge_list.emplace_back(e);
67          else
68              std::cerr << "Vertex out of bounds" << std::endl;
69      }
70
71      auto outgoing_edges(size_t v) const ->vector<Edge<T>>
72      {
73          std::vector<Edge<T>> edges_from_v;
74          for (auto &e :const Edge<T> & : edge_list)
```

```
        {
            if (e.src == v)
                edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Step 5: Depth-First Search function
template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest) ->vector<unsigned long long>
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

    stack.push( x: 1); // Assume that DFS always starts from vertex ID 1
```

```
99
100         while (!stack.empty())
101         {
102             auto current_vertex :size_t  = stack.top();
103             stack.pop();
104 
105             if (visited.find(current_vertex) == visited.end())
106             {
107                 visited.insert(current_vertex);
108                 visit_order.push_back(current_vertex);
109 
110                 for (auto e :Edge<T>  : G.outgoing_edges(current_vertex))
111                 {
112                     if (visited.find(e.dest) == visited.end())
113                     {
114                         stack.push(e.dest);
115                     }
116                 }
117             }
118         }
119         return visit_order;
120     }
121 
```

```cpp
// Step 6: Create reference graph
template <typename T>
auto create_reference_graph() ->Graph<T>
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};

    for (auto &i :pair<...> & : edges)
        for (auto &j :pair<unsigned long long, T> & : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

// Step 7: Driver & Test function
template <typename T>
void test_DFS()
{
    auto G :Graph<unsigned> = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;
    auto dfs_visit_order :vector<size_t> = depth_first_search(G, dest: 1);
    for (auto v :size_t : dfs_visit_order)
        std::cout << v << std::endl;
}
```

```cpp
#include "ILOB1.h"
#include <iostream>

int main() {
    using T = unsigned;
    test_DFS<T>();
    return 0;
}
```

Output:

```
"C:\Users\Vince Urbano\Documents\ILO-B1-GRAPH\cr
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
```

ILO B.2:

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

// Step 2: Edge struct
template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};
```

```cpp
// Step 3: Stream operator overload
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e :Edge<T> & : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
}

// Step 4: Graph class definition
template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}

```

```
52      auto vertices() const ->size_t
53      {
54          return V;
55      }
56
57      auto &edges() const ->const vector<Edge<T>> &
58      {
59          return edge_list;
60      }
61
62      void add_edge(Edge<T> &&e)
63      {
64          if (e.src >= 1 && e.src <= V &&
65              e.dest >= 1 && e.dest <= V)
66              edge_list.emplace_back(e);
67          else
68              std::cerr << "Vertex out of bounds" << std::endl;
69      }
70
71      auto outgoing_edges(size_t v) const ->vector<Edge<T>>
72      {
73          std::vector<Edge<T>> edges_from_v;
74          for (auto &e :const Edge<T> & : edge_list)
75          {
76              if (e.src == v)
77              {
```

```
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

// Step 5: Reference graph setup
template <typename T>
auto create_reference_graph() ->Graph<T>
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;

    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};

    for (auto &i :pair<...> &  : edges)
        for (auto &j :pair<unsigned long long, T> &  : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});

    return G;
}

// Step 6: Breadth-First Search function
template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest) ->vector<unsigned long long>
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;

```

```
122
123         queue.push( x: 1); // Assume that BFS always starts from vertex ID 1
124
125         while (!queue.empty())
126         {
127             auto current_vertex :size_t  = queue.front();
128             queue.pop();
129
130             if (visited.find(current_vertex) == visited.end())
131             {
132                 visited.insert(current_vertex);
133                 visit_order.push_back(current_vertex);
134
135                 for (auto e :Edge<T>  : G.outgoing_edges(current_vertex))
136                     queue.push(e.dest);
137             }
138         }
139         return visit_order;
140     }
141
```

```
142     // Step 7: Driver & Test function
143     template <typename T>
144     void test_BFS()
145     {
146         auto G :Graph<unsigned>  = create_reference_graph<unsigned>();
147         std::cout << G << std::endl;
148
149         std::cout << "BFS Order of vertices: " << std::endl;
150         auto bfs_visit_order :vector<size_t>  = breadth_first_search(G,  dest: 1);
151         for (auto v :size_t  : bfs_visit_order)
152             std::cout << v << std::endl;
153     }
```

```cpp
#include "ILOB2.h"
#include <iostream>

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

```
"C:\Users\Vince Urbano\Documents\ILO-B2\cmake-build-de
1:      {2: 2}, {5: 3},
2:      {1: 2}, {5: 5}, {4: 1},
3:      {4: 2}, {7: 3},
4:      {2: 1}, {3: 2}, {5: 2}, {6: 4}, {8: 5},
5:      {1: 3}, {2: 5}, {4: 2}, {8: 3},
6:      {4: 4}, {7: 4}, {8: 1},
7:      {3: 3}, {6: 4},
8:      {4: 5}, {5: 3}, {6: 1},

BFS Order of vertices:
1
2
5
4
8
3
6
7

Process finished with exit code 0
```

COMPUTER ENGINEERING

QUEZON CITY

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.

Answer:
Depth-First Search (DFS) is the most helpful algorithm. It goes as deep as possible along each branch before backtracking to explore other paths from the same starting vertex.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

Answer:

3. In the performed code, what data structure is used to implement the Breadth First Search?

Answer:
A Queue (First-In, First-Out / FIFO) is used to implement BFS.

4.How many times can a node be visited in the BFS?
A node can only be visited once. A standard BFS keeps track of visited nodes using a set or boolean array so each node is visited/processed exactly once.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini to assist me with learning how the code works.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.



I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.